Homework Assignment 2

Q1 and Q2, You can find solutions directly from Lec02 and Lec03 PPT.

Problem 1. (15 points) What is the output of the following queries.

An example of how to answer this question:

```
select distinct name
from    instructor
order by name  desc
```

Answer: List in alphabetic descending order the names of all instructors.

(a) (5 points)

```
select name
from instructor
where salary > some (select salary
                     from instructor
                     where dept name = 'Biology');
```

Answer: Find names of instructors with salary greater than that of some (at least one) instructor in the Biology department.

(b) (10 points)

```
select distinct S.ID, S.name
from student as S
where not exists ( (select course_id
                    from course
                    where dept_name = 'Biology')
                   except
                    (select T.course_id
                     from takes as T
                     where S.ID = T.ID));
```

For (b), **also explain** what is the output of the **first nested query**, what is the output of the **second nested query**, and what are **"except" and "not exists".**

Answer: Find all students (ID and name) who have taken all courses offered in the Biology department. First nested query lists all courses offered by Biology department. Second nested query lists all courses a student took. "except" is set difference operation (-). "not exists " returns true if the set is empty; if the set is not empty, returns false.

Problem 2. (10 points) Is the following SQL query Correct? If not, why? And how to fix it.

select *dept_name, ID,* **avg** (*salary*)
from *instructor*
group by *dept_name*;

Answer: This is NOT correct. Attributes in ***select*** *clause outside of aggregate functions must appear in* ***group by*** *list. To fix it:*

select *dept_name, ID,* ***avg*** *(salary)*
from *instructor*
group by *dept_name, ID;*

Problem 3. (25 points) Consider the following relational database where the primary keys are underlined:

student(<u>student_id</u>, name)

course(<u>course_id</u>, classroom)

take(<u>student_id, course_id</u>)

(a) (10 points) Give an SQL data definition of the above database (i.e., write SQL commands that create tables). Integrity constraints (primary key and foreign keys) must be included in the data definition.

```
create table student (
student_id char(10),
name varchar(30),
primary key (student_id));

create table course (
course_id char(10),
classroom char(10),
primary key (course_id));

create table take (
student_id char(10),
course_id char(10),
primary key (student_id, course_id),
foreign key (student_id) references student,
foreign key (course_id) references course);
```

(b) (10 points) Write an SQL query that finds the IDs of the courses that John Smith has taken. (Hint: You need tables: student and take.)

Select distinct course_id

From student, take

Where name = 'John Smith' and student.student_id = take.student_id;

(c) (5 points) Write an SQL query that finds the classrooms of the courses that John Smith has taken. (Hint: You need all three tables.)

Select distinct classroom

From student, take, course

Where name = 'John Smith' and student.student_id = take.student_id and course.course_id = take.course_id;

Problem 4. (25 points) Consider the following relational database where the primary keys are

underlined:

employee(<u>id</u>, name, team name, annual salary)

project(<u>project_name</u>, budget)

team(<u>team_name</u>, manager_id) // manager id is a foreign key to the employee table

works_on(<u>team_name, project_name</u>)

Express each of the following queries in SQL.

(a) (5 points) Find the total amount of budget to support projects contributed by team 'Y'. (Hint: You need two table, and an aggregate function.)

```
select sum(budget)
from project natural join works_on
where team_name = 'Y'
```

or

```
select sum(budget)
from project, works_on
where team_name = 'Y' and project.project_name = works_on.project_name
```

(b) (10 points) Find the names of employees who earn more than any (all) participant in project 'Z'. (Hint: You may want to use a subquery and "all" operation.)

```
select name
from employee
where annual_salary > all (
  select annual_salary
  from employee natural join works_on
  where project_name = 'Z')
```

(c) (10 points) Find the names of the teams that do not participate in any project. (Hint: You may want to use "not in" operation and a subquery.)

```
select team_name
from team
where team_name not in (select team_name from works_on)
```


Problem 5. (10 points) You can finish this question using our Lab environment and PostgreSQL.

Check the Appendix if you forget how to run PostgreSQL and the pokemon database in our csci366 server.

(a) (10 points) Write an SQL query to find type_1 and the average speed (rename it as "avg_speed") of each type_1 whose average speed is greater than 70. (Hint: You should use an aggregate function , "group by" and "having". You should ignore type_2.). Your output should be:

```
  type_1  |       avg_speed
----------+----------------------
 Dragon   |  83.0312500000000000
 Electric |  84.5000000000000000
 Normal   |  71.5510204081632653
 Psychic  |  81.4912280701754386
 Fire     |  74.4423076923076923
 Flying   | 102.5000000000000000
 Dark     |  76.1612903225806452
(7 rows)
```

```
pokemon_jxie=> select type_1, avg(speed) as avg_speed
pokemon_jxie-> from pokemon
pokemon_jxie-> group by type_1
pokemon_jxie-> having avg(speed) > 70;
```

(b) (10 points) Write an SQL statement to find the number of distinct pokemons whose speed is greater than 145.

The result should be:

```
 count
-------
     6
(1 row)
```

```
pokemon_jxie=> select count(name)
pokemon_jxie-> from pokemon
pokemon_jxie-> where speed > 145;
```

Explain the difference between your solution to 5(b) and the following query:

select count(name), speed
from pokemon
where speed >145
group by speed;

The difference is, our solution to 5(b) count all pokemons with speed > 145.
But the given query grouped pokemon using their speed and count each group's number.
e.g. pokemons with speed 150 will be in one group, so the count is 4 meaning there are 4 pokemons with speed 150.

Problem 6. (15 points) Use recursive SQL to find all managers (names) of David in the following relation named **emp**.

```
with recursive emp_man as(select managerid as m from emp where employeename ='David'
                          union all
                          select managerid
                          from emp_man, emp where emp_man.m = emp.employeeid)
select employeename from emp where employeeid in (select * from emp_man);
```